

Résumé TP SQL – DDL & DML

BDD-1CS | ESI 2025-2026

Partie 1 : DDL | Partie 2 : DML

Partie 1 : DDL (Data Definition Language)

1.1 Exploration de la Metabase

```
SHOW DATABASES;
```

Rôle : Affiche toutes les bases de données disponibles sur le serveur MySQL.

```
+-----+
| Database          |
+-----+
| information_schema |
| mysql             |
| performance_schema |
| sys               |
+-----+
```

```
USE INFORMATION_SCHEMA;
SELECT * FROM SCHEMATA;
```

Rôle : USE sélectionne une base. SCHEMATA liste les bases avec leur encodage et collation.

```
+-----+-----+-----+
| CATALOG_NAME | SCHEMA_NAME          | DEFAULT_CHARACTER_SET_NAME |
+-----+-----+-----+
| def          | mysql                | utf8mb4                    |
| def          | information_schema    | utf8mb3                    |
| def          | performance_schema    | utf8mb4                    |
| def          | sys                  | utf8mb4                    |
+-----+-----+-----+
```

```
SELECT VERSION();
```

Rôle : Affiche la version du serveur MySQL installé.

```
+-----+
| VERSION() |
+-----+
| 9.7.0     |
+-----+
```

1.2 Création de la Base de Données

```
CREATE DATABASE AIRLINE;
USE AIRLINE;
```

Rôle : CREATE DATABASE crée une nouvelle base. USE la sélectionne pour travailler dedans.

```
Query OK, 1 row affected (0.185 sec)
Database changed
```

1.3 Création des Tables

```
CREATE TABLE IF NOT EXISTS PLANE (
  RefPlane SMALLINT PRIMARY KEY,
  Company CHAR(15) NOT NULL,
  Capacity SMALLINT CHECK(Capacity > 0),
  Constructor VARCHAR(30)
);
```

Rôle : Crée la table PLANE avec une clé primaire, une contrainte NOT NULL et une contrainte CHECK.

```
CREATE TABLE IF NOT EXISTS FLIGHT (
  IdFlight INT PRIMARY KEY,
  IDPlane SMALLINT,
  airportD TEXT(30),
  airportA TEXT(30),
  FOREIGN KEY (IDPlane) REFERENCES PLANE(RefPlane)
);
```

Rôle : Crée la table FLIGHT avec une clé étrangère référençant PLANE.

```
SHOW TABLES;
```

Rôle : Affiche toutes les tables de la base courante.

```
+-----+
| Tables_in_airline |
+-----+
| flight            |
| plane            |
+-----+
```

```
DESC FLIGHT;
DESCRIBE PLANE;
```

Rôle : DESC et DESCRIBE sont identiques – affichent la structure d’une table (colonnes, types, clés, valeurs par défaut).

```
-- DESC FLIGHT
+-----+-----+-----+-----+-----+-----+
| Field      | Type       | Null    | Key    | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| IdFlight   | int        | NO      | PRI    | NULL    |       |
| IDPlane    | smallint   | YES     | MUL    | NULL    |       |
| airportD   | tinytext   | YES     |        | NULL    |       |
| airportA   | tinytext   | YES     |        | NULL    |       |
+-----+-----+-----+-----+-----+-----+
```

1.4 Contraintes d'Intégrité Référentielle

Option	Comportement lors d'un DELETE sur PLANE
(défaut / NO ACTION)	Bloqué – erreur si un vol référence l'avion
ON DELETE CASCADE	Supprime automatiquement les vols associés
ON DELETE SET NULL	Met IDPlane à NULL dans FLIGHT, le vol reste
ON DELETE SET DEFAULT	Non supporté par InnoDB – se comporte comme NO ACTION

Exemple – Comportement par défaut (erreur) :

```
INSERT INTO PLANE VALUES (1, 'AIR ALGERIE', 400, 'AIRBUS'),
                          (2, 'LUFTHANSA', 450, 'BOEING');
INSERT INTO FLIGHT VALUES (1, 1, 'HOUARI BOUMEDIENE', 'FRANKFURT');
DELETE FROM PLANE WHERE REFPLANE=1;
```

```
ERROR 1451 (23000): Cannot delete or update a parent row:
a foreign key constraint fails ('airline'. 'flight',
CONSTRAINT 'flight_ibfk_1' FOREIGN KEY ('IDPlane')
REFERENCES 'plane' ('RefPlane'))
```

Exemple – CASCADE :

```
FOREIGN KEY (IDPlane) REFERENCES PLANE(RefPlane)
ON UPDATE CASCADE ON DELETE CASCADE
-- Après DELETE FROM PLANE WHERE REFPLANE=1 :
SELECT * FROM FLIGHT; -- Resultat : vide
```

```
Empty set (0.005 sec) -- Le vol a ete supprime automatiquement
```

Exemple – SET NULL :

```
FOREIGN KEY (IDPlane) REFERENCES PLANE(RefPlane)
ON UPDATE SET NULL ON DELETE SET NULL
-- Après DELETE FROM PLANE WHERE REFPLANE=1 :
SELECT * FROM FLIGHT;
```

```
+-----+-----+-----+-----+
| IdFlight | IDPlane | airportD          | airportA |
+-----+-----+-----+-----+
|          1 |      NULL | HOUARI BOUMEDIENE | FRANKFURT |
+-----+-----+-----+-----+
-- IDPlane mis a NULL, le vol existe toujours
```

1.5 Modification de Structure – ALTER TABLE

```
ALTER TABLE FLIGHT ADD COLUMN DURATION INT;
```

Rôle : Ajoute une nouvelle colonne DURATION de type INT.

```
ALTER TABLE FLIGHT ALTER COLUMN DURATION SET DEFAULT 2;
```

Rôle : Définit la valeur par défaut de DURATION à 2.

```
| DURATION | int | YES | | 2 | | -- Default = 2
```

```
ALTER TABLE FLIGHT DROP COLUMN DURATION;
```

Rôle : Supprime la colonne DURATION de la table.

Différences entre CHANGE, MODIFY et RENAME :

Commande	Syntaxe	Rôle
CHANGE	CHANGE ancien nouveau TYPE	Renomme ET change le type
MODIFY	MODIFY nom TYPE	Change le type uniquement
RENAME COLUMN	RENAME COLUMN ancien TO nouveau	Renomme uniquement

```
ALTER TABLE FLIGHT CHANGE IDPlane RefPlane SMALLINT;
-- IDPlane renommé en RefPlane, type reste SMALLINT

ALTER TABLE FLIGHT MODIFY RefPlane SMALLINT;
-- Redéfinit le type de RefPlane (sans renommer)

ALTER TABLE FLIGHT RENAME COLUMN RefPlane TO IDPlane;
-- Renomme RefPlane en IDPlane sans changer le type
```

1.6 Gestion des Contraintes avec ALTER

```
ALTER TABLE FLIGHT ADD COLUMN DURATION INT;
ALTER TABLE FLIGHT ADD CONSTRAINT C1 CHECK (DURATION > 1);
INSERT INTO FLIGHT VALUES(2,1,'HOUARI BOUMEDIENE','FRANKFURT', 1);
```

Erreur : DURATION=1 viole la contrainte C1 (DURATION doit être > 1).

```
ERROR 3819 (HY000): Check constraint 'C1' is violated.
```

```
ALTER TABLE FLIGHT DROP CONSTRAINT C1; -- Supprime la contrainte
ALTER TABLE FLIGHT DROP COLUMN DURATION; -- Supprime la colonne
```

1.7 Gestion des Tables

```
CREATE TABLE TAB(A SMALLINT); -- Cr e une table simple
ALTER TABLE TAB RENAME TO TEST; -- Renomme la table
DROP TABLE TEST; -- Supprime d finitivement la table
```

Partie 2 : DML (Data Manipulation Language)

2.1 INSERT – Deux syntaxes

```
-- Syntaxe 1 : ordre des colonnes obligatoire
INSERT INTO PLANE VALUES (1,'AIR ALGERIE',400,'AIRBUS'),
                          (2,'LUFTHANSA',450,'BOEING'),
                          (3,'AMERICAN AIRLINE',500,'BOEING'),
                          (4,'AIRBUS',290,'ALITALIA'),
                          (5,'BOING',3000,'GERMAN WINGS');

-- Syntaxe 2 : ordre des colonnes libre
INSERT INTO PLANE (REFPLANE,COMPANY,CONSTRUCTOR,CAPACITY)
VALUES (6,'ALITALIA','AIRBUS',290);
```

Différence :

- **Syntaxe 1** : les valeurs doivent suivre *exactement* l'ordre des colonnes de la table.
- **Syntaxe 2** : on spécifie les colonnes explicitement, l'ordre peut être différent.

RefPlane	Company	Capacity	Constructor
1	AIR ALGERIE	400	AIRBUS
2	LUFTHANSA	450	BOEING
3	AMERICAN AIRLINE	500	BOEING
4	AIRBUS	290	ALITALIA
5	BOING	3000	GERMAN WINGS
6	ALITALIA	290	AIRBUS

2.2 Insertions avec erreurs

```
INSERT INTO FLIGHT VALUES (3,10, 'HOUARI BOUMEDIENE', 'FRANKFURT');
```

Erreur : IDPlane=10 n'existe pas dans PLANE ⇒ violation de clé étrangère.

```
ERROR 1452 (23000): Cannot add or update a child row:
a foreign key constraint fails
```

2.3 UPDATE

```
UPDATE PLANE SET CAPACITY=2000 WHERE REFPLANE=2;
```

Rôle : Modifie la capacité de l'avion n°2 (LUFTHANSA) à 2000.

```
Rows matched: 1  Changed: 1  Warnings: 0
| 2 | LUFTHANSA | 2000 | BOEING | -- Capacite mise a jour
```

2.4 DELETE

```
DELETE FROM FLIGHT;
```

Rôle : Supprime toutes les lignes de FLIGHT (sans condition = tout supprimer).

```
Query OK, 2 rows affected
Empty set -- Table vide apres suppression
```

2.5 SELECT simples

```
SELECT 'Ahmed' AS Nom, 45 AS Age;
```

Rôle : Affiche des valeurs littérales sans accéder à une table.

```
+-----+-----+
| Nom    | Age  |
+-----+-----+
| Ahmed  | 45   |
+-----+-----+
```

```
SELECT CURRENT_DATE , CURRENT_TIME;
```

Rôle : Affiche la date et l'heure actuelles du serveur.

```
+-----+-----+
| CURRENT_DATE | CURRENT_TIME |
+-----+-----+
| 2026-04-26   | 12:01:08     |
+-----+-----+
```

2.6 SELECT avec DISTINCT

```
SELECT DISTINCT Constructor FROM PLANE;
```

Rôle : DISTINCT élimine les doublons – affiche chaque constructeur une seule fois.

```
+-----+
| Constructor |
+-----+
| AIRBUS      |
| BOEING      |
| ALITALIA    |
| GERMAN WINGS|
+-----+
4 valeurs uniques au lieu de 6 lignes
```

2.7 SELECT avec WHERE et conditions

```
SELECT * FROM PLANE WHERE Constructor='BOEING';
```

Rôle : Filtre les avions dont le constructeur est exactement 'BOEING'.

```
SELECT * FROM PLANE WHERE Capacity>2000 AND Constructor='BOEING';
```

Rôle : Deux conditions combinées avec AND. Résultat vide car BOEING a Capacity=2000 (pas > 2000).

```
Empty set -- 2000 n'est pas > 2000
```

```
SELECT * FROM PLANE WHERE Constructor LIKE 'BO_I%';
```

Rôle : LIKE avec jokers : _ = 1 caractère quelconque, % = 0 ou plusieurs caractères. Trouve BOEING (BO-E-I-NG).

```
| 2 | LUFTHANSA      | 2000 | BOEING |
| 3 | AMERICANAIRLINE | 500  | BOEING |
```

```
SELECT * FROM PLANE WHERE Capacity IN (2000, 2500);
```

Rôle : IN vérifie si la valeur est dans une liste. Trouve seulement Capacity=2000.

```
| 2 | LUFTHANSA | 2000 | BOEING |
```

```
SELECT * FROM PLANE WHERE Capacity BETWEEN 2000 AND 3000;
```

Rôle : BETWEEN inclut les bornes (2000 et 3000 compris).

```
| 2 | LUFTHANSA | 2000 | BOEING      |
| 5 | BOING      | 3000 | GERMAN WINGS |
```

```
SELECT * FROM PLANE WHERE Capacity IS NULL;
SELECT * FROM PLANE WHERE Capacity IS NOT NULL;
```

Rôle : IS NULL / IS NOT NULL vérifient si une valeur est ou non NULL (on ne peut pas utiliser = NULL).

```
IS NULL      --> Empty set (aucune capacite vide)
IS NOT NULL  --> Les 6 avions (toutes capacites renseignees)
```

Récapitulatif des Commandes DDL/DML

Commande	Rôle
CREATE DATABASE	Crée une nouvelle base de données
USE	Sélectionne la base à utiliser
CREATE TABLE	Crée une nouvelle table avec ses colonnes et contraintes
DROP TABLE	Supprime définitivement une table
ALTER TABLE ADD	Ajoute une colonne ou contrainte
ALTER TABLE DROP	Supprime une colonne ou contrainte
ALTER TABLE CHANGE	Renomme une colonne et/ou change son type
ALTER TABLE MODIFY	Modifie le type d'une colonne
ALTER TABLE RENAME	Renomme une colonne ou une table
INSERT INTO	Insère des lignes dans une table
UPDATE	Modifie des valeurs existantes
DELETE	Supprime des lignes
SELECT	Interroge et affiche des données
WHERE	Filtre les résultats selon une condition
DISTINCT	Élimine les doublons dans les résultats
LIKE	Recherche avec motif (_ = 1 char, % = plusieurs)
IN	Vérifie si valeur est dans une liste
BETWEEN	Vérifie si valeur est dans un intervalle (bornes incluses)
IS NULL / IS NOT NULL	Vérifie la présence ou absence de valeur NULL

Résumé réalisé lors des TP SQL – BDD-1CS ESI 2025-2026